

USDT0 · SECURITY REVIEW

USDT0

Polygon Upgrade.

A security review of Polygon Upgrade.

CLIENT

USDT0

PROJECT

Polygon Upgrade

AUDITORS

Guardian

AUDIT WINDOW

May 12 – May 29,
2025

FINAL REPORT

June 5, 2025

§ 00

Contents.

36 pages — overview, confidence ranking, scope, 25 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 24 findings	P. 09
	Remediation Review · 1 finding	P. 34
§ 05	Disclaimer	P. 35
§ 06	About Guardian	P. 36

§ 01

Overview.

Guardian reviewed Polygon Upgrade across 2 review rounds. The final report includes 5 Low, 20 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	Everdawn-Labs/pos-portal
CLIENT	METHODOLOGY
USDT0	Smart contract · Manual · Main Review · Remediation Review
PROJECT	
Polygon Upgrade	
PREPARED BY	
Guardian	
AUDIT WINDOW	
May 12 – May 29, 2025	
FINAL REPORT	
June 5, 2025	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	0	0	0	0	0
Low	5	0	3	0	2
Informational	20	0	11	1	8
All severities	25	0	14	1	10

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Polygon Upgrade in `Everdawn-Labs/pos-portal`. The scope includes 563 source lines across 15 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
contracts/common/AccessControlMixin.sol	15	18
contracts/common/ContextMixin.sol	18	28
contracts/common/EIP712Base.sol	50	70
contracts/common/Initializable.sol	12	15
contracts/common/NativeMetaTransaction.sol	69	96
contracts/common/Proxy/IERCProxy.sol	3	4
contracts/common/Proxy/Proxy.sol	29	38
contracts/common/Proxy/UpgradableProxy.sol	81	102
contracts/child/ChildToken/IChildToken.sol	3	4
contracts/child/ChildToken/UpgradeableChildERC20/ERC20.sol	100	328
contracts/child/ChildToken/UpgradeableChildERC20/UChildERC20.sol	32	43
contracts/child/ChildToken/UpgradeableChildERC20/UChildERC20Proxy.sol	8	10
contracts/child/ChildToken/DappTokens/IERC7802.sol	5	20
contracts/child/ChildToken/DappTokens/UChildUSD0.sol	115	165
contracts/child/ChildToken/DappTokens/WithBlockedList.sol	23	51

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
L-01	Old Signatures Become Invalid	Informational	LOW	ACKNOWLEDGED	p. 9
L-02	Missing ERC-165 Support	Best Practices	LOW	RESOLVED	p. 10
L-03	Use Of Vanilla ecrecover Allows Signature Malleability	Validation	LOW	ACKNOWLEDGED	p. 11
L-04	UChildUSDT0 Not EIP-2612 Compliant	Logical Error	LOW	RESOLVED	p. 12
L-05	Lack Of Smart Wallet Support In permit	Logical Error	LOW	ACKNOWLEDGED	p. 13
I-01	Inconsistent Use Of Revert Messages	Best Practices	INFORMATIONAL	RESOLVED	p. 14
I-02	Missing Event On setOFTContract Update	Best Practices	INFORMATIONAL	RESOLVED	p. 15
I-03	Typo In EIP712Base	Best Practices	INFORMATIONAL	RESOLVED	p. 16
I-04	Missing SPDX-License Identifier	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-05	isContract Probably Soon Redundant	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 18
I-06	Unnecessary Looping	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 19
I-07	No Initialization Protection	Validation	INFORMATIONAL	RESOLVED	p. 20
I-08	Removal Of Function Can Break Integrations	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 21
I-09	Misleading Comment In crosschainMint	Best Practices	INFORMATIONAL	RESOLVED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-10	Missing Storage Gaps	Informational	INFORMATIONAL	RESOLVED	p. 23
I-11	Grouped Issues: Inherited Rom Polygon's pos-portal Contracts	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 24
I-12	OpenZeppelin v3.4.0 Recommendation	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 26
I-13	Unnecessary String Concatenation	Gas Optimization	INFORMATIONAL	RESOLVED	p. 27
I-14	Recover Tokens Stuck In USDT Contract	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 28
I-15	Invalid Test Should Be Removed	Informational	INFORMATIONAL	RESOLVED	p. 29
I-16	Warning About In Flight Bridges	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 30
I-17	Floating Pragma	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 31
I-18	Missing Validation During Upgrade	Validation	INFORMATIONAL	PARTIALLY RESOLVED	p. 32
I-19	LZ Config Block Confirmations Is Too Low	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 33

REMEDIATION REVIEW

I-01	Events Should Be Grouped Together	—	INFORMATIONAL	ACKNOWLEDGED	p. 34
------	-----------------------------------	---	---------------	--------------	-------

LOW **L-01**

ACKNOWLEDGED

Old Signatures Become Invalid

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team2/blob/6a81fd7529dcb5a70116d0c0326fdc5ae23da5e1/contracts/child/ChildToken/DappTokens/UChildUSDToken.sol#L49-L50
REVIEW	Main Review

— DESCRIPTION

With the USDT0 upgrade, the token name and domain separator will be updated. As a result, signed meta txs according to the old domain separators, will become invalid immediately after the update.

This could cause UX issues for users utilizing DEXes for gasless swaps, for example.

If DEXes are not made aware of this domain separator change, they may continue using the old one, resulting in failed swaps for users.

— RECOMMENDATION

Consider documenting this behavior and informing users that they must sign new permits after the USDT0 upgrade.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

Missing ERC-165 Support

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The EIP 7802 states: The inclusion of ERC-165 provides an additional security check for integrators. By providing the interface identifier through the supportsInterface method, callers can programmatically confirm that the token adheres to the IERC7802 interface.

Current UChildUSDt0 implementation inherits from IERC7802, supporting crosschain ERC20 transfers. However, this interface does not enforce IERC165, different from the current OFT extension deployments in other chains.

RECOMMENDATION

Consider adding the supportsInterface function to correctly signal the EIP support, similar to the current deployments:

```
function supportsInterface(bytes4 interfaceId) external pure override returns (bool) {  
    return interfaceId == type(IERC7802).interfaceId || interfaceId == type(IERC165).interfaceId;  
}
```

See <https://eips.ethereum.org/EIPS/eip-7802>

RESOLUTION

Status: Resolved.

LOW **L-03**

ACKNOWLEDGED

Use Of Vanilla ecrecover Allows Signature Malleability

CATEGORY Validation

LOCATION <https://github.com/Everdawn-Labs/pos-portal/blob/4ec7849ffef561c3b92f71dbe6eb5ab9020169e8/contracts/child/ChildToken/DappTokens/UChildUSDT0.sol#L62>

REVIEW Main Review

DESCRIPTION

USDT uses vanilla `ecrecover` for signer recovery, which is susceptible to malleability due to signature variations.

This does not cause any immediate damage since the signed permit itself does not change. However, this should still be considered for improvement.

RECOMMENDATION

Consider using OpenZeppelin's `ECDSA` library for signer recovery to mitigate malleability risks

RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

UChildUSDT0 Not EIP-2612 Compliant

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team1/blob/6a81fd7529dcb5a70116d0c0326fdc5ae23da5e1/contracts/common/NativeMetaTransaction.sol#L26
REVIEW	Main Review

DESCRIPTION

According to the EIP-2612, compliant contracts must implement 3 new functions in addition to EIP-20:

```
function permit(address owner, address spender, uint value, uint deadline, uint8 v, bytes32 r, bytes32 s) external
```

```
function nonces(address owner) external view returns (uint)
```

```
function DOMAIN_SEPARATOR() external view returns (bytes32)
```

However, `UChildUSDT0` has a `nonces` internal mapping, and uses the `getNonce` public getter instead. Consequently, the expected `permit` functionality will be prevented for many wallets and tools that rely on the standard `nonces(address)` getter.

RECOMMENDATION

Consider making the `nonces` mapping public to comply with the EIP.

RESOLUTION

Status: Resolved.

LOW **L-05**

ACKNOWLEDGED

Lack Of Smart Wallet Support In `permit`

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team2/blob/master/contracts/child/ChildToken/DappTokens/UChildUSDt0.sol#L62
REVIEW	Main Review

DESCRIPTION

The `permit` function uses `ecrecover` for signature verification, which only supports Externally Owned Accounts (EOAs). This excludes smart contract wallets (e.g., Gnosis Safe, DAOs) that rely on ERC-1271 for validating signatures.

USDt0's deployments on Arbitrum and Optimism already support ERC-1271 by using OpenZeppelin's `SignatureChecker`, which delegates to `isValidERC1271SignatureNow` when the signer is a contract. The Polygon version has not adopted this approach.

This discrepancy breaks cross-chain UX. Smart wallet users who can interact with USDt via `permit` on Arbitrum or Optimism are blocked when bridging to Polygon, where the same flow fails.

As account abstraction adoption grows (e.g., EIP-7702 in Pectra), supporting contract wallets is critical. The current implementation limits composability and prevents protocols like CoW Swap from offering gasless approvals to smart wallet users.

Note: Using `SignatureChecker` also makes the contract EIP-7377 safe, should it be approved in future. See [discussion](<https://github.com/OpenZeppelin/openzeppelin-contracts/issues/4855>) and [commit](<https://github.com/OpenZeppelin/openzeppelin-contracts/pull/4951/files>)

RECOMMENDATION

Update the `permit` function to use `SignatureChecker`, as done on Arbitrum and Optimism, to enable ERC-1271 compatibility and ensure cross-chain consistency.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

RESOLVED

Inconsistent Use Of Revert Messages

CATEGORY	Best Practices
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

`UChildUSDT0` contains several error messages starting with `UChildUSDT0:` while some others use `TetherToken:` (i.e. `multiTransfer`) and `USD70:` (`AccessControlMixin`). This may create some confusion to offchain services, as it might imply it's a different contract.

RECOMMENDATION

Consider maintaining a consistent error message structure throughout the contract

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

Missing Event On setOFTContract Update

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team2/blob/master/contracts/child/ChildToken/DappTokens/UChildUSDT0.sol#L158>

REVIEW Main Review

DESCRIPTION

The `setOFTContract` function updates the OFT contract address, a critical change that should be clearly recorded on-chain. However, no dedicated event is emitted to signal this update. The only emitted event is `RoleGranted`, which is generic and does not specifically indicate that the OFT contract has changed.

For comparison, USDT0 implementations on other chains like Arbitrum emit a dedicated `LogSetOFTContract` event when this change is made.

RECOMMENDATION

Emit a specific event (e.g., `LogSetOFTContract`) in the `setOFTContract` function to explicitly record the new OFT contract address for better transparency and traceability.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

RESOLVED

Typo In EIP712Base

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team2/blob/6a81fd7529dcb5a70116d0c0326fdc5ae23da5e1/contracts/common/EIP712Base.sol#L23>

REVIEW Main Review

DESCRIPTION

The comment on line 23 of the EIP712Base contract contains a typo:

"... the contractsa that inherits ..." should be corrected to "... the contracts that inherit ...".

Additionally, the comment format is not consistent with the rest of the codebase, which uses the NatSpec style with `/**` and `*/`.

RECOMMENDATION

Update the comment.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Missing SPDX-License Identifier

CATEGORY Best Practices
LOCATION Global
REVIEW Main Review

DESCRIPTION

All contracts in the scope, except `IERC7802.sol`, `ERC20.sol`, and `WithBlockedList.sol`, are missing an SPDX-License Identifier.

RECOMMENDATION

Consider adding SPDX license identifiers to these contracts.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-05**

ACKNOWLEDGED

isContract Probably Soon Redundant

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team1/blob/6a81fd7529dcb5a70116d0c0326fdc5ae23da5e1/contracts/common/Proxy/UpgradableProxy.sol#L91-L101
REVIEW	Main Review

— DESCRIPTION

The `isContract` check used in the `UpgradableProxy.updateImplementation` is probably soon redundant with the Pectra update as EOAs can contain code with EIP-7702.

Currently, there is a proposal to implement Pectra on Polygon although not accepted yet:
<https://forum.polygon.technology/t/pip-61-pectra-eips/20783>

— RECOMMENDATION

As the issue is present in the proxy contract, it can't be updated. If Polygon implements Pectra, extra care should be taken when updating to a new implementation, making sure the correct address is set, as the `isContract` check will not work as expected.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06

ACKNOWLEDGED

Unnecessary Looping

CATEGORY Best Practices

LOCATION <https://github.com/Everdawn-Labs/pos-portal/blob/4ec7849ffef561c3b92f71dbe6eb5ab9020169e8/contracts/child/ChildToken/DappTokens/UChildUSDT0.sol#L158>

REVIEW Main Review

DESCRIPTION

As per our discussions with the USDT team, only one assignment is expected to hold true for

`DEPOSITOR_ROLE`

Therefore, loops in `_set0FTContract` or `oftContract` are unnecessary and could be replaced with a single revoke and assignment.

RECOMMENDATION

Consider removing the loop and adding a single revoke and assignment to improve code readability and eliminate unnecessary complexity.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-07

RESOLVED

No Initialization Protection

CATEGORY	Validation
LOCATION	https://github.com/Everdawn-Labs/pos-portal/blob/4ec7849ffef561c3b92f71dbe6eb5ab9020169e8/contracts/child/ChildToken/DappTokens/UChildUSDT0.sol#L12
REVIEW	Main Review

DESCRIPTION

`upgradeToUSDT0` is not access-controlled, meaning anyone can call it.

Since an equivalent to `disableInitializers` is not implemented, anyone could call the method on the implementation contract itself and create a parallel identity of the USDT0 token.

For the real USDT0 token, this is not an issue, as the context is via the proxy. However, given the scale of USDT operations, the possibility of such ambiguity should be addressed.

RECOMMENDATION

Consider adding a `disableInitializers` equivalent to `UChildUSDT0` to prevent misuse of the implementation contract.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-08**

ACKNOWLEDGED

Removal Of Function Can Break Integrations

CATEGORY Informational
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The update of the USDT contract removes the function `withdraw` which was previously used to bridge out of Polygon. This may cause unexpected behavior for wallets or applications which relied on calling this function in their smart contracts.

During the review, a check on all addresses that have previously called `withdraw` was performed and no smart contracts were found.

— RECOMMENDATION

Consider documenting this risk for users.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09

RESOLVED

Misleading Comment In crosschainMint

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team2/blob/6a81fd7529dcb5a70116d0c0326fdc5ae23da5e1/contracts/child/ChildToken/DappTokens/UChildUSDToken.sol#L72>

REVIEW Main Review

DESCRIPTION

The `crosschainMint` function includes a comment stating: "Make sure minting is done only by this function."

However, there is also a separate `mint` function that can be called by the admin, allowing them to mint any amount of USDT0 tokens to any address. This contradicts the comment above.

RECOMMENDATION

If the `crosschainMint` function is intended to be the sole method for minting tokens, the admin-only `mint` function should be removed.

If that's not the case, the comment in `crosschainMint` should be updated to accurately reflect the intended behavior.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-10**

RESOLVED

Missing Storage Gaps

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team2/blob/master/contracts/child/ChildToken/UpgradeableChildERC20/UChildERC20.sol , https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team2/blob/master/contracts/child/ChildToken/DappTokens/WithBlockedList.sol
REVIEW	Main Review

DESCRIPTION

The `UChildUSDT0` contract inherits from both `UChildERC20` and `WithBlockedList`. However, neither of these parent contracts defines a storage gap. This omission poses a risk for future upgrades.

If additional storage variables are introduced in `UChildERC20` or `WithBlockedList`, they may overwrite existing storage slots, potentially colliding with variables such as `USDT0_VERSION` declared in `UChildUSDT0`.

RECOMMENDATION

To ensure safe and upgradeable contract design, add storage gaps (e.g., `uint256[50] private __gap;`) to both `UChildERC20` and `WithBlockedList`.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-11

ACKNOWLEDGED

Grouped Issues: Inherited From Polygon's pos-portal Contracts

CATEGORY	Informational
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

These issues are present in contracts inherited or utilized by USDT from Polygon's `pos-portal` suite. We advise the USDT team to review and address them based on their prioritization.

Note that issues in `UpgradeableProxy.sol` cannot be fixed without changing the deployment address and are included for awareness only. Other implementation contracts can be modified if deemed important.

UpgradeableProxy.sol

1. Missing Double-Step Ownership Transfer

Ownership transfer lacks a secure two-step pattern.

Recommendation: Document current behavior and validate new owner address thoroughly before transfer.

2. **No Return Value Forwarding in `updateAndCall` **

The function does not forward return data from the delegate call.

Recommendation: Be aware that return values from called functions will be discarded.

3. ** `updateAndCall` Alters `msg.sender` Context **

Typically, `upgradeToAndCall`-style functions directly delegatecall the implementation. However, `UpgradeableProxy` of pos-portal performs a regular `call` to itself, which routes through the fallback and only then delegatecalls the implementation.

This changes `msg.sender` in the implementation to the proxy contract itself (`address(this)`), not the proxy owner. Many meta-transaction patterns treat `msg.sender == address(this)` as an indicator of a meta-tx and attempt to extract the real sender (`msgSender`) from calldata. Since `updateAndCall` does not append any such data, `msgSender` resolves to the zero address.

Recommendation:

Be aware of this behavior when using `updateAndCall`. If the implementation expects meta-tx structure, it may misinterpret the sender as `address(0)`.

I-11 Grouped Issues: Inherited Rom Polygon's pos-portal Contracts

CONTINUED

EIP712Base.sol

- ****Typo in `_setDomainSeperator` ****

The word "Seperator" is misspelled; should be "Separator."

Recommendation: Rename to `_setDomainSeparator`. As it's internal, this change won't break external dependencies.

NativeMetaTransaction.sol

1. ****Unnecessary `payable` Modifier****

`executeMetaTransaction` is marked `payable` but doesn't handle native currency.

Recommendation: Remove `payable` to prevent accidental loss of native tokens. If retained for gas saving reasons, clearly document the behavior.

2. **Mismatch Between Comment and Code**

The function comment states that relayer address is appended to calldata, but this isn't reflected in the implementation.

Recommendation: Either update the comment to match current behavior or modify the code accordingly.

3. **No Deadline or Expiry on Meta Transactions**

Meta-transactions that fail do not increment the user's nonce, leaving it unused and publicly available.

Since there's no deadline parameter (like in permit), any party can replay this transaction indefinitely as long as the nonce is unchanged.

to calldata, but this isn't reflected in the implementation.

Recommendation: Consider adding a deadline parameter to limit replayability and mitigate long-term replay risks.

— RECOMMENDATION

NA

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-12**

ACKNOWLEDGED

OpenZeppelin v3.4.0 Recommendation

CATEGORY Informational
LOCATION Global
REVIEW Main Review

DESCRIPTION

The current codebase uses Solidity version 0.6.6 and OpenZeppelin Contracts v3.1.0. To improve security and stability, we refer to the [official recommendation](<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/CHANGELOG.md#400-2021-03-23>) from OpenZeppelin, which states that version 3.4.0 is the latest and most appropriate version for use with Solidity 0.6.x.

RECOMMENDATION

Update OpenZeppelin Contracts from v3.1.0 to v3.4.0.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-13

RESOLVED

Unnecessary String Concatenation

CATEGORY Gas Optimization

LOCATION <https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team1/blob/6a81fd7529dcb5a70116d0c0326fdc5ae23da5e1/contracts/child/ChildToken/DappTokens/UChildUSDToken.sol#L19>

REVIEW Main Review

DESCRIPTION

The `upgradeToUSDT0` updates some configurations with the new `USD₯0` name, including the revert message for role access control:

```
_setupContractId(string(abi.encodePacked("USD₯0")));
```

However, this line was used in the `UChildERC20` to concatenate "Child" string with the token symbol:

```
_setupContractId(string(abi.encodePacked("Child", symbol_)));
```

As there is no concatenation needed, the plain string can be passed as an argument to this function.

RECOMMENDATION

Remove the encoding and send plain "USD₯0" string as parameter:

```
_setupContractId("USD₯0");
```

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-14**

ACKNOWLEDGED

Recover Tokens Stuck In USDT Contract

CATEGORY	Informational
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Approximately \$800,000 worth of tokens are currently stuck in the USDT contract, likely due to users mistakenly transferring tokens directly to the contract address. Since the contract does not include a mechanism to recover these funds, they remain irretrievable.

— RECOMMENDATION

Consider implementing a recovery mechanism to allow retrieval of mistakenly sent tokens. This could follow the approach taken by USDC, which includes a controlled `recoverERC20` function.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-15**

RESOLVED

Invalid Test Should Be Removed

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team2/blob/master/test/UChildUSDT0Fork.t.sol#L665
REVIEW	Main Review

DESCRIPTION

In `UChildUSDT0Fork.t.sol`, the test `testInitializeReverts` fails to compile because the `UChildUSDT0` contract does not implement the function `initialize`.

RECOMMENDATION

Remove the test `testInitializeReverts`.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-16

ACKNOWLEDGED

Warning About In Flight Bridges

CATEGORY	Informational
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

In the current implementation, Ethereum to Polygon bridges are completed when the deposit function is called by the `childChainManager`. After the update, bridges will be completed using the `crosschainMint` function.

Bridges cannot be completed on Polygon if the update occurs while there are in-transit bridge requests. The update should only be performed once all requests have been processed.

RECOMMENDATION

It has been stated that the Polygon team will handle the migration process and ensure that all previous messages are settled.

USDT Team should also consider consulting with Polygon team over unlocking of USDT locked in root manager on mainnet.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-17**

ACKNOWLEDGED

Floating Pragma

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team2/blob/6a81fd7529dcb5a70116d0c0326fdc5ae23da5e1/contracts/child/ChildToken/DappTokens/IERC7802.sol#L2
REVIEW	Main Review

DESCRIPTION

The `IERC7802` contract uses Solidity version `^0.6.0`, while all the other contracts in the codebase use the fixed `0.6.6` version.

RECOMMENDATION

Consider setting the pragma version to `0.6.6` in the `IERC7802` contract, similar to the rest of the codebase.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-18**

PARTIALLY RESOLVED

Missing Validation During Upgrade

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team1/blob/6a81fd7529dcb5a70116d0c0326fdc5ae23da5e1/contracts/child/ChildToken/DappTokens/UChildUSD0.sol#L12
REVIEW	Main Review

DESCRIPTION

The `upgradeToUSD0` address parameters are crucial for the correct migration to USD0. Although this function is executed along with the implementation update and called by the proxy owner, some minimal validations should be implemented to guarantee no mistakes are made:

- `oftContract = address(0)`
- `newAdmin = address(0)`
- `decimals() = oft.sharedDecimals()`

Keep in mind the `ArbitrumExtensionV2` also has zero address check for the OFT address, so these validations will be consistent with other migrations.

RECOMMENDATION

Consider adding the minimal validations when executing the upgrade function.

RESOLUTION

Status: Partially resolved.

INFORMATIONAL **I-19**

ACKNOWLEDGED

LZ Config Block Confirmations Is Too Low

CATEGORY	Configuration
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The current Layer Zero configuration requires a minimum amount of block confirmations the OApp needs to wait in order to send or receive messages. In case of USDT0, this value will be set to 256 blocks.

However, this value seems low with the fact that there have been 157 block reorgs in the past in Polygon. Additionally, other protocols like Stargate and Abracadabra have this value set to 512 blocks.

— RECOMMENDATION

Consider increasing the Layer Zero block confirmation value to 512.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Events Should Be Grouped Together

CATEGORY —

LOCATION <https://github.com/GuardianOrg/pos-portalusdt0polygonupgrade-team2/blob/guardian-fixes-david/contracts/child/ChildToken/DappTokens/UChildUSDT0.sol#L12>

REVIEW Remediation Review

DESCRIPTION

The `LogSet0FTContract` was recently added to the UChildUSDT0 contract. However, it was placed at the top of the contract (Line 12) while the rest of events are at the bottom of the contract.

RECOMMENDATION

Group the events together at the top of the contract where events are typically found.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.